claude-windows / a9fd4426-f234-4e4a-bfb4-1f2c8255c661

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\a9fd4426-f234-4e4a-bfb4-1f2c8255c661\subagents\workflows\wf_076beba8-59c\agent-a4394146867d444cd.jsonl`
- SHA-256: `b4ddbf3c724aedde3d9c4c439572bc63895ad045b5f417d09d93fd6e5b0e8bd0`
- Source modified: `2026-05-31T04:18:53+00:00`
- Imported at: `2026-07-05T16:48:20+00:00`
- project: `wf_076beba8-59c`
- session_id: `a9fd4426-f234-4e4a-bfb4-1f2c8255c661`

Transcript

1. user (2026-05-31T04:17:58.782Z)

Source Extractor

Research question: "Identify the best LOCAL (self-hosted, single NVIDIA CUDA GPU) vision / document-AI / OCR models ? and, as a clearly-flagged approval-gated fallback only, paid cloud API services ? for parsing BANK STATEMENT PDFs into structured transaction tables (date, narration/description, withdrawal/debit, deposit/credit, running balance), for a LOCAL-FIRST personal-finance tool ("muneem").

Target documents: Indian bank statements (HDFC, Axis, ICICI, AU Small Finance) ? dense, multi-column, often SEMI-RULED or borderless tables, sometimes multiple accounts per statement; both born-digital (text layer present) and the harder cases where pdfplumber's extract_tables/coordinate clustering is unreliable (e.g. Axis, whose column layout varies between statements).

HARD CONSTRAINTS (a model that violates these is unusable for us):
1. MUST run 100% LOCALLY for statement contents (highly sensitive). NO cloud APIs by default. Paid/cloud is acceptable ONLY as an explicitly-approved fallback ? still report the best cloud options but label them clearly as "cloud/sensitive ? approval-gated, not default".
2. MUST be usable from Python (transformers / vLLM / Ollama / ONNX Runtime / native lib).
3. License MUST be permissive & commercially safe for BOTH the CODE and the MODEL WEIGHTS: MIT, BSD, Apache-2.0, ISC, HPND. AVOID and explicitly FLAG anything GPL/LGPL/AGPL, CC-BY-NC / non-commercial, research-only, or custom "community"/acceptable-use licenses with restrictions. This is critical ? many document-AI models and their training datasets are non-commercial (e.g. LayoutLMv3 weights are CC-BY-NC; Surya and Marker use GPL-or-commercial dual licensing; several VLM weights ship custom restrictive licenses). Verify the WEIGHTS license, not just the repo license.
4. Hardware: a single consumer/prosumer NVIDIA GPU. Give VRAM tiers (~8-12 GB consumer, ~16-24 GB) and which models/quantizations fit each.

Evaluate and compare these LOCAL approaches ? for each give: code license, MODEL-WEIGHTS license (state explicitly), latest release / maintenance status as of 2025-2026, VRAM need + quantization options, Python integration path, and reliability specifically on DENSE FINANCIAL TABLES / multi-column statements:
- Open vision-language models (VLMs) for document/table understanding: Qwen2-VL and Qwen2.5-VL (note per-size license ? which sizes are Apache-2.0 vs restricted), MiniCPM-V 2.6, InternVL2 / 2.5, Microsoft Phi-3.5-vision, Mistral Pixtral, Meta Llama-3.2-Vision (flag the Llama Community License restrictions), Moondream2, GOT-OCR2.0, olmOCR and other OCR-specialized VLMs.
- OCR-specialist / document-AI pipelines: PaddleOCR + PP-Structure (table recognition), docTR, EasyOCR, Tesseract (already used), Microsoft Table Transformer / TATR (check code AND PubTables-1M dataset/weights license), Donut, Surya (FLAG license), Marker (FLAG license), MinerU / PDF-Extract-Kit, RapidOCR.
- Table-structure recognition specifically (cell/grid reconstruction for borderless tables): TATR, PP-Structure, UniTable, etc.

Also assess: for SEMI-RULED / borderless bank tables, is a prompt-driven VLM (page image -> JSON

transactions) more reliable than a classic OCR + table-structure pipeline? What are the accuracy/hallucination risks of VLMs on financial NUMBERS (transposed digits, dropped/duplicated rows), and how should output be VERIFIED? (muneem already machine-checks parses by balance reconciliation ? opening + sum(deposits) - sum(withdrawals) == closing, plus per-row running-balance walks and cross-checks against the bank's printed summary ? so any model's output can be automatically validated, which should inform the recommendation.)

Finally, briefly cover the best PAID/CLOUD fallbacks (clearly flagged approval-gated for sensitive data): AWS Textract (AnalyzeExpense / Tables), Google Document AI, Azure AI Document Intelligence, Reducto, LlamaParse, Anthropic/OpenAI vision ? note any self-hosting option and data-handling terms.

Deliver a RANKED shortlist of the most promising LOCAL options for muneem (NVIDIA GPU, Python, commercial-license-safe), each with license (code + weights), VRAM tier, maturity, strengths/weaknesses for dense bank-statement tables, and a concrete recommendation: which local model(s) to prototype for muneem's statement parsing and how they'd combine with the existing deterministic parsers + balance-reconciliation verification (e.g. VLM as a fallback/cross-check when deterministic parsing fails to reconcile). Cite sources and call out any license uncertainty explicitly."

Fetch and extract key claims from this source:
URL: https://github.com/allenai/olmocr
Title: olmOCR (Allen AI) ? Apache-2.0 OCR VLM, repo + PyPI + arXiv
Found via: OCR-specialist VLMs & dense Indian bank statements (focused/technical) search

Task

1. Use WebFetch to retrieve the page content.
2. Assess source quality: primary research/institution? secondary reporting? blog/opinion? forum? unreliable?
3. Extract 2-5 FALSIFIABLE claims that bear on the research question. Each claim must:
   - be a concrete, checkable statement (not vague generalities)
   - include a direct quote from the source as support
   - be rated central/supporting/tangential to the research question
4. Note publish date if available.

If the fetch fails or the page is irrelevant/paywalled, return claims: [] and sourceQuality: "unreliable".

Structured output only.

2. assistant (2026-05-31T04:18:02.275Z)

I'll fetch the olmOCR GitHub page to extract claims relevant to the research question about local document-AI models for bank statement parsing.

3. user (2026-05-31T04:18:17.901Z)

olmOCR Local Document Processing Analysis

1. Licenses

Code License: "Apache-2.0" (explicitly stated in repository)

Model Weights License: Not explicitly specified in provided content. The repository references model releases on Hugging Face (e.g., `allenai/olmOCR-2-7B-1025-FP8`) but does not state license terms for model weights in the excerpt provided.

2. Hardware Requirements

- **GPU:** "Recent NVIDIA GPU (tested on RTX 4090, L40S, A100, H100) with at least 12 GB of GPU RAM"
- **Storage:** "30GB of free disk space"

- **Quantization:** FP8 quantization available; default model is "olmOCR-2-7B-1025-FP8"
- **Inference optimization:** Optional flashinfer library for faster inference

3. Python Integration

- **PyPI Package:** `olmocr` (installable via `pip install olmocr[gpu]`)
- **Inference Backend:** vLLM-based (switched from sglang as of v0.1.75)
- **Remote Option:** OpenAI API-compatible endpoint support
- **CLI Tool:** `olmocr` command-line executable

4. Base Model & Size

- **Architecture:** Vision Language Model (VLM), 7 billion parameters
- **Base Model:** References "Qwen2.5-VL" in training code section
- **Latest Release:** v0.4.0 (October 21, 2025); v0.4.27 noted as latest

5. Document Capabilities

Supported Types: "PDF, PNG, and JPEG based documents"

Claimed Features:
- "Convert into text with a natural reading order, even in the presence of figures, multi-column layouts, and insets"
- "Support for equations, tables, handwriting, and complex formatting"
- "Automatically removes headers and footers"

Table Extraction: Explicitly mentions table support; benchmark includes "Tables" category

6. Maintenance Status

- **Latest Version:** v0.4.27 (March 12, 2026)
- **Recent Updates:** v0.4.0 released October 21, 2025
- **Status:** Active development with regular releases and benchmark suite

7. Benchmark Numbers

olmOCR-Bench Overall Score: v0.4.0 achieves "82.4±1.1" overall on comprehensive test suite covering 7,000+ test cases across 1,400 documents

Specific Categories (v0.4.0):
- Tables: 84.9
- Multi-column: 47.7
- Headers & footers: 96.1

Note on Bank Statements: While the toolkit explicitly supports tables and numeric content, no specialized claims about banking PDFs or financial statement accuracy are stated in the provided content.

4. assistant (2026-05-31T04:18:20.711Z)

The model weights license wasn't captured. Since the research question is critical on weights licensing, let me verify it on the Hugging Face model card, which is the authoritative source for weights license.

5. user (2026-05-31T04:18:26.860Z)

Model License and Details

License

Apache 2.0 - The exact license statement from the page: "This model is licensed under Apache 2.0. It is intended for research and educational use in accordance with Ai2's [Responsible Use Guidelines](https://allenai.org/responsible-use)."

Base Model

Qwen/Qwen2.5-VL-7B-Instruct - The model was fine-tuned from this base model and additionally fine-tuned using GRPO RL training.

Model Size

7B parameters (note: the model card references "8B params" in one section, but the model name and documentation consistently refer to the 7B base model from Qwen2.5-VL)

Key Details

- This is a quantized FP8 version of olmOCR-2-7B-1025
- Quantization performed using llmcompressor
- Fine-tuned on the [olmOCR-mix-1025](https://huggingface.co/datasets/allenai/olmOCR-mix-1025) dataset
- Further optimized with GRPO RL training for math equations, tables, and challenging OCR cases

6. user (2026-05-31T04:18:28.671Z)

olmOCR License Analysis

Code License

The code is licensed under **Apache 2.0**. The README states: **"olmOCR is licensed under Apache 2.0. A full copy of the license can be found on GitHub."**

Model Weights License

The README does **not include a separate statement** about model weights licensing or the base model license. There is no explicit mention of Qwen2.5-VL's license terms.

Base Model

The code overview references **"SFT Finetuning code for Qwen2.5-VL"** as a reusable component, indicating Qwen2.5-VL serves as the foundation model. However, the README provides no information about whether the base model carries an Apache-2.0 license or any other license designation.

Summary: Only the code itself is explicitly licensed under Apache 2.0; model/weights licensing is not addressed in this README.

7. assistant (2026-05-31T04:18:36.062Z)

I now have authoritative confirmation of the weights license from the Hugging Face model card. Both code and weights are Apache-2.0, and the base model is Qwen2.5-VL-7B-Instruct (also Apache-2.0). I have enough to extract the falsifiable claims.

Key findings:
- olmOCR is a primary source (Allen AI / Ai2, a non-profit research institution) ? the official GitHub repo plus the canonical HF model card.
- Both code AND weights are Apache-2.0 ? this is the critical fact for muneem's license-safety constraint. The model card adds "intended for research and educational use in accordance with Ai2's Responsible Use Guidelines" ? a non-binding usage guideline, not a license restriction, but worth flagging as mild uncertainty.
- Base model is Qwen2.5-VL-7B-Instruct (itself Apache-2.0).
- ~12 GB VRAM minimum, FP8 quantization available, default model is FP8.
- Python via `pip install olmocr[gpu]`, vLLM backend.
- Targets PDF/PNG/JPEG; explicitly supports tables, multi-column, equations; olmOCR-Bench Tables score 84.9, overall 82.4.
- Actively maintained (v0.4.x releases through early 2026).

8. user (2026-05-31T04:18:46.243Z)

Structured output provided successfully

9. assistant (2026-05-31T04:18:53.836Z)

Structured output delivered. Summary of what I found on olmOCR (Allen AI):

- Source quality: primary ? official Allen AI/Ai2 GitHub repo plus the canonical Hugging Face model card.
- License (the critical fact for muneem): both code and the 7B weights are Apache-2.0. I verified the weights license directly on the HF model card (`allenai/olmOCR-2-7B-1025-FP8`) because the GitHub README only covered the code. The base model, Qwen2.5-VL-7B-Instruct, is also Apache-2.0, so the full stack is commercially safe. One mild caveat: the model card adds a non-binding "intended for research and educational use in accordance with Ai2's Responsible Use Guidelines" note ? a usage guideline, not a license restriction.
- Fit for muneem: runs locally on a single NVIDIA GPU (?12 GB VRAM; tested RTX 4090/L40S/A100/H100), default model is FP8-quantized 7B, installable via `pip install olmocr[gpu]` on a vLLM backend.
- Tables/financial reliability: explicitly supports tables and multi-column layouts, RL-tuned for tables and hard OCR, scores 84.9 on the Tables category of olmOCR-Bench (82.4 overall). However, there are no claims specific to bank/financial-statement numeric accuracy ? reinforcing that muneem's balance-reconciliation verification should gate any VLM output.